Testing và CI: Ghi chú

Team Task Management · Tổng hợp từ Task 5 (BE + FE + CI) · 27/10/2026

1. Nguyên tắc chung
2. Test Backend (vitest + supertest + Prisma)
3. Test Frontend (RTL + MSW + user-event)
4. CI với GitHub Actions
5. Bảng lỗi đã gặp
6. Checklist trước khi push

1. Nguyên tắc chung

> **Quy tắc số 1:** test chỉ có giá trị khi nó **đỏ đúng lúc code sai**. Viết xong một test, hãy cố ý làm hỏng code (xoá một dòng, đổi `if (isLoading)` thành `if (isLoading || true)`, bỏ `return` ...). Nếu test vẫn xanh thì test đó không kiểm tra gì cả.

- **Test hành vi, không test chi tiết cài đặt.** BE thì kiểm tra status + body của API. FE thì kiểm tra những gì người dùng thấy và làm được (chữ, nút, link).
- **Mỗi test độc lập.** Không phụ thuộc thứ tự chạy hay dữ liệu test trước để lại. BE dọn DB trước mỗi test, FE tạo QueryClient mới cho mỗi test.
- **Arrange → Act → Assert.** Chuẩn bị dữ liệu, mock → thao tác, gọi API → kiểm tra.
- **Code nằm ngoài `it()` không phải là test.** Vitest chỉ báo kết quả cho những gì nằm trong `it()` / `test()`.
- **Kiểm tra nội dung, không chỉ kiểm tra "có tồn tại".** `<h2>` lúc nào cũng có. Điều cần kiểm tra là nó chứa "Team A".
- **TDD khi sửa bug hoặc thêm hành vi:** viết test trước, thấy test đỏ, sửa code, thấy test xanh.

2. Test Backend

2.1 Database riêng cho test

- Dùng DB `test_taskflow`, tách khỏi DB dev `taskflow`. Test sẽ xoá sạch dữ liệu nên **không bao giờ** được chạy trên DB dev.
- **Guard** ở đầu `test/setup.ts`: chỉ chạy khi `NODE_ENV === "test"` **và** tên DB đúng là `test_taskflow`. Lấy tên DB bằng cách parse URL (`new URL(DATABASE_URL).pathname.slice(1)`), không dùng `includes()`.
- Trước mỗi test: `TRUNCATE TABLE ... RESTART IDENTITY CASCADE`.
  - `RESTART IDENTITY`: id bắt đầu lại từ 1.
  - `CASCADE`: không phải quan tâm thứ tự khoá ngoại.
  - Khi thêm model mới vào schema thì nhớ thêm vào danh sách `TABLES`.
- `afterAll`: `prisma.$disconnect()`.
- `fileParallelism: false`: các file test dùng chung một DB, nếu chạy song song sẽ xoá dữ liệu của nhau.

2.2 Nạp biến môi trường: vì sao đặt trong `vitest.config.ts`

```
// vitest.config.ts
test: {
  env: config({ path: ".env.test" }).parsed,
  setupFiles: ["./test/setup.ts"],
  fileParallelism: false,
}
```

- **Import hoisting:** trong ESM, mọi `import` chạy **trước** các dòng code khác trong file. Nếu gọi `dotenv.config()` ở đầu file test, thì `env.ts` đã được import và đã đọc `process.env` từ trước đó, nên đọc phải giá trị cũ (DB dev).
- **dotenv không ghi đè** biến đã tồn tại. `import "dotenv/config"` trong `env.ts` nạp `.env`, nhưng không đè lên các biến mà vitest đã đặt sẵn từ `.env.test`.

2.3 Token: dùng thật, không fake

- Helper `createUserAndLogin()` đăng ký và đăng nhập qua API thật để lấy access token. Nhờ vậy test đi qua đúng middleware xác thực như khi chạy thật.
- Dùng một biến đếm để tạo email khác nhau cho mỗi user, tránh lỗi trùng email.
- Helper giúp test ngắn và dễ đọc: `createWorkspace(token, input)`, `expectFieldError(res, "name")`.

2.4 Các trường hợp cần có cho mỗi endpoint

Trường hợp	Mong đợi	Ghi chú
Không gửi token	401	
Body sai (rỗng, toàn khoảng trắng, quá dài)	400 + `errors.<field>`	Kiểm tra cả field nào bị lỗi
Param sai (`/workspaces/abc`)	400	Params là chuỗi: dùng `z.coerce`
Đúng thành viên nhưng sai role	403	MEMBER làm việc chỉ ADMIN được làm
Người ngoài workspace	**404**, không phải 403	Không để lộ là workspace có tồn tại
IDOR: id của tài nguyên thuộc workspace khác	404	Luôn kiểm tra quan hệ cha từ URL
Field lạ trong body (`projectId: 999`)	Bị bỏ qua	zod tự loại bỏ key không khai báo
Sau khi xoá mềm	GET chi tiết trả 404, không còn trong danh sách	Kiểm tra cả count, ví dụ `projectCount`
Thành công	201 / 200 / 204 + body đúng	204 thì không có body, và không bị treo

> **Lỗi thường gặp:** `res.status(204)` mà không gọi `.send()` nên request bị **treo**. Trong Express 5, cứ `throw new AppError(...)` là lỗi sẽ được bắt tự động.

3. Test Frontend

3.1 Cài đặt

```
// vite.config.ts
/// <reference types="vitest/config" />    ← thiếu dòng này thì TS báo 'test' does not exist
test: { environment: "jsdom", setupFiles: ["./src/test/setup.ts"] }

// src/test/setup.ts
import "@testing-library/jest-dom/vitest";
beforeAll(() => server.listen({ onUnhandledRequest: "error" }));
afterEach(() => { server.resetHandlers(); cleanup(); });
afterAll(() => server.close());
```

`renderWithProviders` bọc component cần test bằng các provider mà app thật cũng dùng:

- **`QueryClient` mới cho mỗi test**, để cache không rò từ test này sang test khác. Đặt `retry: false` để test lỗi không phải chờ retry.
- **`MemoryRouter`** giả lập URL (`route`). Kèm `<Routes><Route path={path}>` để `useParams` đọc được giá trị. Nếu không có `Route` , `useParams()` trả về `{}` và `Number(undefined)` ra `NaN` .
- Đặt `path` mặc định là `"*"` . Nếu để `"/"` mà quên truyền `path` , trang sẽ trắng trơn mà không báo lỗi gì.

3.2 Chọn query: getBy, queryBy hay findBy

Query	Khi không tìm thấy	Dùng khi
`getBy…`	Ném lỗi ngay	Phần tử **có ngay** tại thời điểm gọi. Là hàm đồng bộ, không cần `await` .
`queryBy…`	Trả về `null`	Khẳng định phần tử **KHÔNG có**: `expect(queryBy…).not.toBeInTheDocument()`
`findBy…`	Ném lỗi sau 1000ms	**Chờ** phần tử xuất hiện, ví dụ dữ liệu từ API. Phải `await` .

Sai

```
expect(await
findByRole("dialog"))
  .not.toBeInTheDocument();
// findBy ném lỗi

await waitFor(async () =>
  expect(await
findByRole(...)));  // chờ 2
lần

await screen.getByText(...);
// getBy không cần await
```

Đúng

```
expect(queryByRole("dialog"))
  .not.toBeInTheDocument();

await screen.findByRole(...);   // findBy đã tự chờ

// Chờ phần tử biến mất:
await waitFor(() =>

expect(queryByRole("dialog")).not.toBeInTheDocument());
```

> **Lần render đầu tiên luôn ở trạng thái loading.** Request là bất đồng bộ, nên dù server trả lời ngay (0ms) thì `isLoading` vẫn là `true` ở lần render đầu. Hệ quả là `getBy…` gọi ngay sau `render()` chỉ thấy skeleton. Muốn kiểm tra dữ liệu, nút hay lỗi đến từ API thì phải dùng `findBy…` .

3.3 Test trạng thái loading (skeleton)

Kiểm tra **quá trình chuyển trạng thái**, không chỉ kiểm tra "có skeleton". Nếu chỉ kiểm tra có skeleton thì test luôn pass, như đã giải thích ở trên.

```
const skeleton = screen.getByTestId("skeleton-list_workspace");   // 1. có ngay
await waitForElementToBeRemoved(skeleton);                        // 2. chờ biến mất
expect(screen.getByText(/don't have any workspace/i)).toBeInTheDocument(); // 3. nội dung
```

- `waitForElementToBeRemoved` cần có `await` . Callback (nếu dùng) phải **trả về phần tử** và **không được `async`** , vì một Promise không bao giờ là `null` .
- Phần tử phải **còn tồn tại** lúc bắt đầu chờ. Nếu nó biến mất quá nhanh (ví dụ dialog đóng ngay) thì chuyển sang cách: `findBy` nội dung mới, sau đó kiểm tra `queryBy` trả về `null` .
- `delay(50)` là đủ. Delay quá 1000ms sẽ vượt thời gian chờ mặc định của `findBy` / `waitFor` .

3.4 MSW: mock API

> **Handler không `return` response** thì MSW chuyển request sang handler tiếp theo. Nếu không handler nào khớp, request bị coi là unhandled và gây **lỗi mạng**. Test lỗi khi đó vẫn xanh, nhưng xanh vì lý do sai.
>
> ```
> http.get(url, () => { new HttpResponse(null, { status: 404 }); })   // ✗ quên return
> ```

- **Gọi `server.use(...)` TRƯỚC khi render hoặc thao tác.** Đăng ký sau khi bấm Submit thì request đã gửi đi rồi.
- **Không đặt `expect` trong handler.** MSW bắt lỗi đó và biến thành lỗi của request, khiến thông báo lỗi khó đọc. Trong handler chỉ **ghi lại** giá trị (`postedBody` , `postCount` , `params`), rồi `expect` ở cuối test.
- **Params trên URL luôn là chuỗi:** `params.workspaceId === "1"` , không phải số `1` .
- **"Database giả":** một mảng dùng chung cho handler GET và POST. POST thêm phần tử vào, GET đọc ra. Nhờ vậy test được việc danh sách cập nhật sau khi tạo.
- **Kiểm tra "không gửi request":** dùng một biến đếm trong handler, rồi `expect(postCount).toBe(0)` .
- `onUnhandledRequest: "error"` trong setup: request nào không có handler sẽ bị báo ngay, không âm thầm đi ra mạng thật.

```
const workspaces: WorkspaceSummary[] = [];
let postedBody: unknown;
server.use(
  http.get("/api/v1/workspaces", () => HttpResponse.json({ data: workspaces })),
  http.post("/api/v1/workspaces", async ({ request }) => {
    postedBody = await request.json();          // ghi lại, expect ở cuối test
    const created = { id: 1, name: "Team A", /* ... */ };
    workspaces.push(created);
    return HttpResponse.json({ data: created }, { status: 201 });
  }),
);
```

3.5 Tìm phần tử và thao tác như người dùng

- **Thứ tự ưu tiên khi tìm phần tử:** `ByRole` (kèm `name`), rồi `ByLabelText`, rồi `ByText`. `ByTestId` chỉ dùng khi không còn cách nào khác, ví dụ skeleton.
- **Kiểm tra qua accessibility:** `toHaveAttribute("aria-invalid", "true")`, `toHaveAccessibleDescription("…")` (lỗi được gắn vào input qua `aria-describedby`), `toHaveAttribute("href", "/")`.
- **`within(dialog)`** giới hạn phạm vi tìm trong một vùng, tránh tìm nhầm phần tử cùng tên ở ngoài.
- **user-event v14:** gọi `const user = userEvent.setup()` ở đầu mỗi test, sau đó `await user.click / type / keyboard("{Escape}")`. Nên dùng thay cho `fireEvent`.
- **Helper** cho các thao tác lặp lại, ví dụ `openCreateDialog(user)` trả về element `dialog`. Đặt helper ở đầu `describe`.

3.6 Test qua page hay test riêng component?

- State nằm ở page (nút mở dialog, state `openDialog`, danh sách được tải lại) thì **test qua page**, giống cách người dùng thao tác thật.
- Component được dùng chung ở nhiều nơi, hoặc có nhiều logic riêng, thì test riêng component và truyền `vi.fn()` cho các props.

3.7 Những bug mà viết test đã giúp phát hiện

- `onOpenChange={(next) => !next && onClose}`: quên gọi hàm, nên bấm Esc không đóng được dialog.
- Thiếu `<Route path>` nên `workspaceId` là `NaN` và API bị gọi tới `/workspaces/NaN`.
- Mọi lỗi đều hiện "Workspace not found". Cần tách: 404 thì hiện "not found" kèm link về danh sách, 500 thì hiện "Try again" (`refetch`).
- Dùng `<Button onClick={navigate}>` để chuyển trang. Nên dùng `<Link>`, vì đó là điều hướng.
- `.map()` trả về phần tử nhưng thiếu `key`.

4. CI với GitHub Actions

4.1 Cấu trúc file: thụt lề quyết định cấp

```
name: CI
on: { push: { branches: [main] }, pull_request: {} }
concurrency: { group: ci-${{ github.ref }}, cancel-in-progress: true }

jobs:
  check:                # tên job
    runs-on: ubuntu-latest
    services:           # ← nằm TRONG job, cùng cấp runs-on / steps
      postgres:
        image: postgres:16
        env: { POSTGRES_USER: ci, POSTGRES_PASSWORD: ci, POSTGRES_DB: test_taskflow }
        ports: ["5432:5432"]          # ← cùng cấp env, KHÔNG nằm trong env
        options: >-
          --health-cmd pg_isready --health-interval 5s --health-timeout 5s --health-retries 10
    env: { NODE_ENV: test, DATABASE_URL: ..., JWT_ACCESS_SECRET: ..., JWT_REFRESH_SECRET: ... }
    steps: [...]
```

Lỗi đã gặp: `services` chỉ thụt 2 dấu cách nên GitHub hiểu đó là một **job** tên "services". `ports` và `options` nằm trong `env` nên Postgres nhận chúng như 2 biến môi trường.

4.2 Thứ tự các bước

#	Bước	Lưu ý
1	`actions/checkout`	Lấy code về máy CI.
2	`actions/setup-node`	`node-version-file: .nvmrc`, để CI dùng cùng phiên bản Node với máy dev. `cache: "npm"`.
3	`npm ci`	Không dùng `npm install`. `npm ci` cài đúng theo `package-lock.json`, và báo lỗi nếu lock lệch với `package.json`.
4	`prisma generate`	Sinh Prisma Client. Thiếu bước này thì typecheck và test ở server đều lỗi.
5	Build `@ttm/shared`	server và ui import package này từ thư mục `dist`.
6	`npm run format:check`	**Không** dùng `format` (`--write`): lệnh đó sửa file trên CI rồi luôn pass.
7	Lint, Typecheck	
8	`prisma migrate deploy`	Tạo bảng trong DB rỗng. Không dùng `migrate dev`, vì lệnh đó có thể hỏi tương tác và tạo migration mới.
9	`npm test`	Chạy test của server (cần Postgres) và ui.
10	`npm run build`	Đảm bảo code build được để deploy.

4.3 Báo hết lỗi trong một lần chạy

```
- name: Install dependencies
  id: install                       # đặt id để các bước sau tham chiếu
  run: npm ci

- name: Lint
  if: ${{ !cancelled() && steps.install.outcome == 'success' }}
  run: npm run lint
```

- Mặc định, một bước fail thì các bước sau **bị bỏ qua**. Ví dụ format fail, bạn sửa xong push lại mới biết lint cũng fail.
- `!cancelled()` : vẫn chạy tiếp dù bước trước fail, nhưng dừng nếu run bị huỷ.
- `steps.install.outcome == 'success'` : nếu bước nền tảng (install, build shared) đã fail thì bỏ qua, vì chạy tiếp chỉ sinh ra lỗi rác.

4.4 Biến môi trường và secret

- Các file `.env*` bị gitignore nên trên CI không có. Phải khai báo biến trong `env:` của job (hoặc của từng step).
- Giá trị **chỉ dùng cho test** (JWT secret giả, mật khẩu Postgres `ci`) được phép ghi thẳng trong file.
- Secret **thật** (production, deploy) phải để trong **GitHub Secrets** và đọc qua `${{ secrets.NAME }}` . Không bao giờ ghi vào file.
- Tên DB của service phải trùng với guard trong `test/setup.ts` (`test_taskflow`). Nếu khác, guard sẽ từ chối chạy test, và đó là guard đang làm đúng việc của nó.
- Phải có `health-cmd` cho service, để các step chỉ chạy khi Postgres đã sẵn sàng.

4.5 `concurrency` : huỷ run cũ

- Push 3 commit liên tiếp thì run 1 và run 2 **có bắt đầu chạy**, nhưng bị huỷ giữa chừng (hiện trạng thái *Cancelled*). Chỉ run 3 chạy hết. Lợi ích: tiết kiệm phút chạy CI, và nhận kết quả của code mới nhất sớm hơn.
- `group: ci-${{ github.ref }}` : mỗi branch hoặc PR là một nhóm riêng, nên các branch khác nhau không huỷ run của nhau.
- **Cẩn thận khi có deploy:** huỷ một bản deploy đang chạy dở trên `main` có thể để hệ thống ở trạng thái nửa vời. Cách hay dùng là chỉ huỷ run của PR:

  ```
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}
  ```

4.6 Kiểm tra CI có thực sự chặn được lỗi

- Cố ý làm hỏng một test, push lên một branch, và CI phải **đỏ**.
- Bật branch protection cho `main` : bắt buộc check CI xanh mới được merge PR.

5. Bảng lỗi đã gặp

Triệu chứng	Nguyên nhân	Cách sửa
Test skeleton pass dù có delay hay không	`getBy` chạy ngay ở lần render đầu, lúc đó luôn đang loading	Kiểm tra cả quá trình: có, rồi biến mất, rồi nội dung

`Unable to find role="button" name /Try again/`	Dùng `getBy` khi lỗi chưa về	Dùng `await findBy…`
Test lỗi pass ngay cả khi xoá `server.use`	Handler không `return`, request unhandled gây lỗi mạng	Luôn `return` response
API bị gọi tới `/workspaces/NaN`	Không có `<Route path>` nên `useParams()` trả về `{}`	Thêm `path` vào `renderWithProviders`
`expected '1' to deeply equal 1`	Params trên URL là chuỗi	So với `"1"`, và `expect` ngoài handler
`findByRole(...).not` luôn fail	`findBy` ném lỗi khi không thấy phần tử	Dùng `queryBy`
`waitForElementToBeRemoved` báo "already removed"	Phần tử đã biến mất trước khi bắt đầu chờ	`findBy` nội dung mới, rồi `queryBy` là `null`
`waitForElementToBeRemoved` không có tác dụng	Thiếu `await`, hoặc callback là `async`	Thêm `await`, bỏ `async`
Submit form trong test không gửi request	Gọi `server.use` sau khi đã thao tác; lấy nút Submit trước khi mở dialog	Đăng ký handler trước; mở dialog rồi mới tìm nút
`'test' does not exist in type UserConfigExport`	Thiếu type của vitest	`/// <reference types="vitest/config" />`
vitest: "No test files found"	Chưa có file `*.test.ts`	Tạo file test cạnh code được test
Test server đọc nhầm DB dev	Gọi `dotenv` trong file test, nhưng import đã chạy trước đó	Đặt `test.env` trong `vitest.config.ts`
Workflow CI không chạy hoặc Postgres không lên	Thụt lề YAML sai cấp	`services` trong job; `ports` và `options` cùng cấp `env`
CI xanh dù code chưa format	Chạy `prettier --write`	Dùng `format:check`

6. Checklist trước khi push

- ☐ `npm run check`: format, lint, typecheck.
- ☐ `npm test`: test của server và ui đều xanh. Máy dùng Node 20 (`nvm use 20`).
- ☐ Mỗi test mới đã được thử làm hỏng code và thấy đỏ ít nhất một lần.
- ☐ Không còn `console.log` hay import không dùng.
- ☐ Mỗi handler MSW đều `return`; mỗi `findBy` / `waitFor` đều có `await`.
- ☐ Thêm model Prisma mới thì đã cập nhật `TABLES` trong `test/setup.ts`, và đã có migration.
- ☐ Thêm biến môi trường mới thì đã khai báo trong `.env.example`, `.env.test` và `env:` của CI.